

Joe Bylund

joseph.bylund@gmail.com • 347-829-5863 • Boston, MA
github.com/jbylund • linkedin.com/in/josephbylund

EXPERIENCE

Klaviyo

Boston, MA

Lead Software Engineer

August 2023 — Present

- **Purchase containment filtering** — designed a bloom filter architecture replacing repeated large-scale ClickHouse scans, bulk-streaming (person, item) pairs into Redis. Reduced data scanned by ~1000x on a path accounting for ~1/3 of total ClickHouse cluster load. Sketched the design for a partner team and shepherded them through implementation (ClickHouse, Redis, python).
- **Information schema as a service** — prototyped a SQL interface over multi-tenant, schema-on-read JSON data, and later rejoined the team that productionized it. Built the manifest service translating user-supplied SQL into queries against JSON-backed tables, giving customers a typed SQL API over untyped storage. Cut manifest resolution latency 3x end to end (python, gRPC, MySQL, StarRocks).
- **Query service** — founding engineer on a gRPC query service over multi-tenant event data: interceptor stack (rate limiting, validation, concurrency limiting, slow-request logging), sharded connection pooling, executors for ClickHouse, PostgreSQL and MySQL, and query rewriting (python, gRPC, ClickHouse, PostgreSQL, MySQL).
- **Metrics service performance** — added profiling instrumentation, then optimized query mapping and result processing, reducing latency ~58% across every request the service serves (python, gRPC, ClickHouse).
- **Custom objects** — built gRPC APIs letting customers define schemas for their own objects, with per-company partitioned rate limiting and a shared Redis Sentinel client, modeled as data vault hubs, links and satellites (python, gRPC, Redis, Pulsar).
- **Developer velocity** — parallelized test containers, added CI cache targets, and built local development environments used across teams.

Altana

New York City (Remote)

Senior Software Engineer

March 2022 — June 2023

- **Business network discovery and traversal** — built tools to construct business networks from graph of world wide shipment data.
- Built search microservice for companies, facilities and transactions using FastAPI, Pydantic, ArangoDB.
- **Arango client library** — built http client for ArangoDB supporting synchronous and asynchronous requests used in search microservice.
- **Spark client library** — shimmed three different client libraries (pyspark, pyhive, databricks-sql-connector) to pep-249 interface, supports creation of business networks.
- **Geocoding client** — writing api client, measuring and improving accuracy, building pipeline.
- **Geocoding pipeline** — built continuous address geocoding pipeline and geocoded ~400 million addresses.

Kensho

Cambridge, MA

Software Engineer

September 2018 — March 2022

- Designed and implemented multi-step document processing pipeline using multiple in-house ML services.
- Designed and built speech-to-text alignment pipeline using SQS & gentle forced aligner.
- Implemented a number of checks in github hooks (pylint, flake8, mypy...), improving the developer experience.

- Migrated fuzzy company identification service to kubernetes and optimized performance.

Moat

New York, NY

Senior Backend Engineer & Data Scientist

September 2013 — September 2018

- **Distributed ETL** — designed and built a fault-tolerant pipeline, cutting cost by an order of magnitude and processing time from ~10 hours to ~1 hour, so data reached clients far earlier in the day (python, SQS, Redis, PostgreSQL).
- **Event routing** — routed ~40 billion events per day from pixel servers into real-time processing, sharding by client, by configurable key and by session, so that one client's traffic could not degrade another's and data arrived pre-aggregated, leaving the ETL far less to merge (c++).
- **Statistical database** — migrated the primary statistical store (500 million rows/day) from non-first to first normal form, improving query latency and throughput while reducing storage demands.
- **Stats API performance** — sped up the primary statistics routes clients called for reporting and exports, cutting latency ~3x and raising the maximum request size through profiling-guided work on serialization and the query paths behind them (PHP, CakePHP).
- **Platform** — migrated users, accounts and metadata from MySQL to PostgreSQL with zero downtime, bridging the two through a MySQL foreign data wrapper during the cutover, and standardized the deployment framework running thousands of servers across ~30 roles on AWS (PostgreSQL, MySQL, EC2, boto3).
- **Data lake** — architected and prototyped a decentralized data lake querying compressed SQLite files on S3 through massively parallel AWS Lambda invocations, and wrote a Multicorn foreign data wrapper exposing it through a standard PostgreSQL interface, so it could be queried like any other data system (AWS lambda, python, SQLite, PostgreSQL).

PROJECTS

- **Sylvan Librarian** (sylvan-librarian.com, [source](#)) — open source Magic: The Gathering card search engine in rust, implementing Scryfall's query syntax and extending it with arithmetic expressions. Self-hosted with blue/green deploys behind nginx; independently forked and deployed to Cloudflare Workers by an outside contributor (rust, python).
- **pg_mimic** — pure-python asyncio implementation of the PostgreSQL wire protocol, letting any python process present a postgres interface to standard clients rather than a bespoke http api. Verified against psycopg, asyncpg, pg8000 and psql (python, asyncio).

OPEN SOURCE CONTRIBUTIONS

Python packaging — pip and packaging

- **Candidate selection** — replaced a linear scan of the supported tag list with a precomputed index lookup, cutting best-candidate selection 11x in profiling, from 68s to 6.2s over 30 calls ([pull request](#)).
- **Tag equality** — short-circuited tag comparison on a cached hash. Comparing tags accounted for two thirds of the time spent picking a candidate; it got 2.5x faster and nearly halved the operation overall ([pull request](#)).

Together these cut the time spent on pip steps in ci (at Kensho).

Python typedshed

Widened the cachetools stubs, where cache sizes were typed as integers although a custom getsizeof may return any number, so correctly typed code using fractional sizes failed type checking ([pull request](#)).

Gnome Shotwell Photo Manager

47 commits from 2013 to 2021, across the import pipeline, thumbnail cache, saved searches, publishing plugins and build system (vala).

- **Import throughput** — reworked thumbnail generation to decode each photo once and derive every thumbnail size from that single decode, rather than decoding once per size ([commit](#)).
- **RAW metadata** — cached metadata on the RAW reader so a file is parsed once per import instead of on every access ([commit](#)).

PHP

Removed a per-column roundtrip to the database in PDO's PostgreSQL driver, resolving the common type OIDs directly instead of querying for each one ([commit](#)). Shipped in PHP since 2015; the saving scales with the width of the result set, and at Moat it cut queries and page load time by more than an order of magnitude on the user administration pages.

EDUCATION

Columbia University

New York, NY

PhD – Computational Chemistry

2007 – 2013

Thesis: Monte-Carlo Sampling of Protein-Ligand Interactions and Computational Improvements to Implicit Solvent Models

- **PLOP** — led development and maintenance of the Protein Local Optimization Program, a molecular mechanics library for protein structure prediction, and wrote its computational mutation scanning module (fortran).
- **Build system** — redesigned it to automatically determine dependencies and take advantage of parallel compilation, reducing build time from ~30 minutes to ~3 minutes and accelerating development.
- **Small molecule database** — built a database representing 95%+ of small molecules in the Protein Data Bank, extending PLOP from a protein-only program to a general molecular mechanics toolkit.
- **Regression testing** — built a Perl based automated test framework, which accelerated development while minimizing bugs and regressions.

Rice University

Houston, TX

Bachelor of Arts – Mathematics

2003 – 2007

Bachelor of Science – Ecology and Evolutionary Biology

TECHNOLOGIES & SKILLS

- **Languages** — Python, SQL, rust, C++
- **Data stores** — ClickHouse, PostgreSQL, MySQL, StarRocks, Redis, ArangoDB; previously Vertica, Redshift
- **Platform** — AWS (EC2, S3, RDS, Lambda, DynamoDB), Kubernetes, gRPC; Kafka, Pulsar, SQS, Kinesis, RabbitMQ

Last updated August 2026